



Universidad
Indoamérica

UNITAT 2 · FONAMENTS DE SISTEMES OPERATIUS

Crides al sistema i E/S de baix nivell

Universidad Tecnológica Indoamérica

Laboratori TD1

Abstracció

- Simplificar i estandarditzar
- Portabilitat del nucli i dels programes (POSIX)
- Facilitar el desenvolupament de controladors
- Estabilitzar l'entorn d'execució

Les abstraccions del curs

Fitxers (U3) · processos i memòria virtual (U4) · comunicació (U5, U6, U8) · fils (U7) · i ara els agents (U9)

Protecció

- Identificació i drets d'ús/accés
- Aïllament entre processos i entre usuaris
- Xifratge de les dades
- Polítiques de seguretat (xarxa, fitxers...)

2026: els mateixos principis s'apliquen a un nou «usuari»: l'**agent d'IA** (identitat pròpia, mínim privilegi, auditoria — U9).

Virtualització

Portabilitat i programació

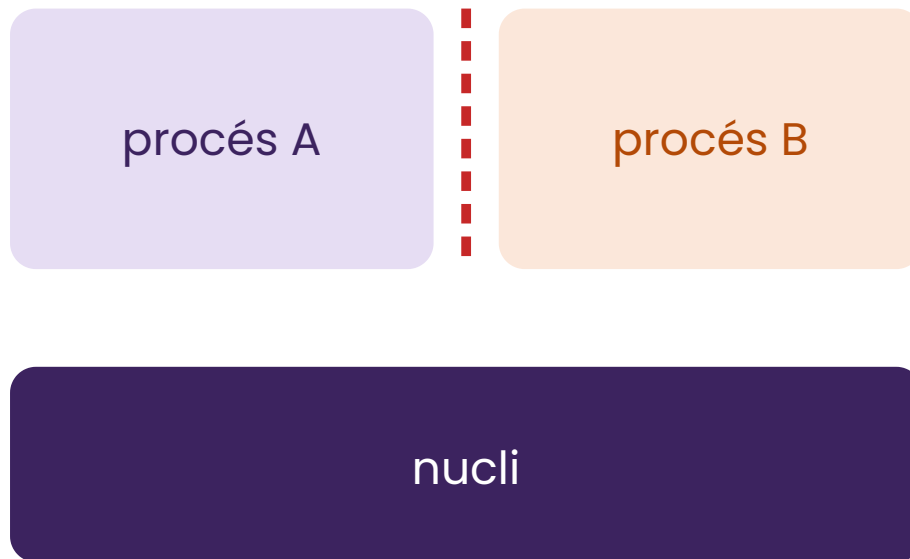
- VFS: una sola interfície per a ext4, Btrfs, NFS, /proc ...
- Una visió homogènia del maquinari (discos USB, SATA i NVMe vistos igual)
- POSIX estandarditza les funcions del sistema

Estabilitat i evolució

- Aïllament de processos: una fallada no fa caure la màquina
- Simular una màquina o un altre SO: QEMU/KVM, WSL 2
- Contenedors: espais de noms + cgroups
- IPv4 sobre IPv6

El concepte de procés

- Cada procés té el seu propi **context**
- Una visió **privada** de la memòria: no veu la dels altres
- Està aïllat i **creu que està sol**: els altres processos i el mateix sistema estan amagats



Dos modes d'execució del processador

Mode usuari (no privilegiat)

El mode per defecte. El programa només suma, llegeix i escriu **dins de la seva pròpia memòria**.

Mode nucli (privilegiat)

Per a qualsevol altra cosa el procés **demana permís** amb una instrucció especial —una crida al sistema—: obrir, modificar o canviar el nom d'un fitxer, assignar memòria, crear o acabar una tasca...

El maquinari canvia de mode i el SO comprova que l'operació estigui permesa: tot el que podria perjudicar altres tasques o usuaris.

El mode nucli i el nucli

- Algunes instruccions (gestió de memòria, interrupcions) estan **prohibides** en mode usuari
- El nucli **gestiona** els processos, però **no és un procés**
- Gestiona els recursos i l'accés al maquinari; sempre resideix a la memòria; es carrega en arrencar (U1, A1.3)
- Un procés d'usuari **passa a mode nucli** per fer servir un servei del nucli

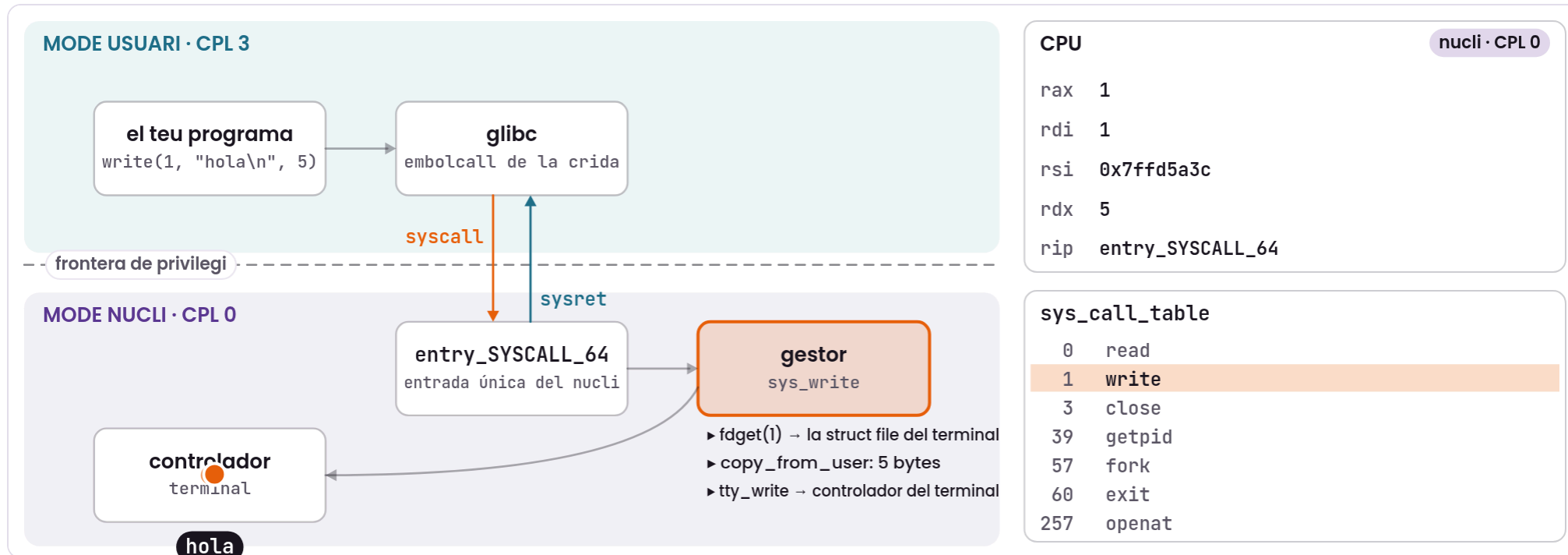
Les crides al sistema: la interfície

- Interfície entre el mode usuari i el mode nucli
- El canvi de mode el dispara una instrucció: `ta` (Sparc), `int 0x80` (x86 antic), avui `syscall` (x86-64), `svc #0` (ARM64), `ecall` (RISC-V)
- El nucli té un gestor d'entrada i una taula de crides

Les crides al sistema són cares!

L'original deia «diversos centenars de cicles». Mesurat a la màquina del curs: **$\approx 0,7 \mu\text{s}$ per iteració** `read` + `write` (milers de cicles).

El viatge de `write(1, "hola\n", 5)`



La instrucció, vista amb objdump

REGISTRE	WRITE(1, BUF, 5)
rax	1 (número de write)
rdi	1 (descriptor)
rsi	adreça de buf
rdx	5

Sortida real

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-  
gnu/libc.so.6 | grep -A6 -E '^[0-9a-f]+'  
<__write_nocancel@@'  
0000000000129060 <__write_nocancel@@GLIBC_PRIVATE>:  
129060: endbr64  
129064: mov    $0x1,%eax  
129069: syscall  
12906b: cmp    $0xffffffffffffffff,000,%rax  
129071: ja     129078  
<__write_nocancel@@GLIBC_PRIVATE+0x18>  
129073: ret
```

Errors: -1, errno i perror

- Una crida que falla retorna **-1** i deixa el codi a **errno**
- Missatge: `void perror(const char *s);` o `%m` a `fprintf` (extensió de glibc)
- Al curs: `exit_if(rc == -1, "write")`

Sortida real

```
$ ./echo blabla < . 1>&0  
write: Bad file descriptor  
[exit status 1]
```

C

```
ssize_t rc = write(STDOUT_FILENO, buf, n);  
if (rc == -1)  
    fprintf(stderr, "write: %m\n");  
exit_if(rc == -1, "write");
```

Traçar les crides: **strace**

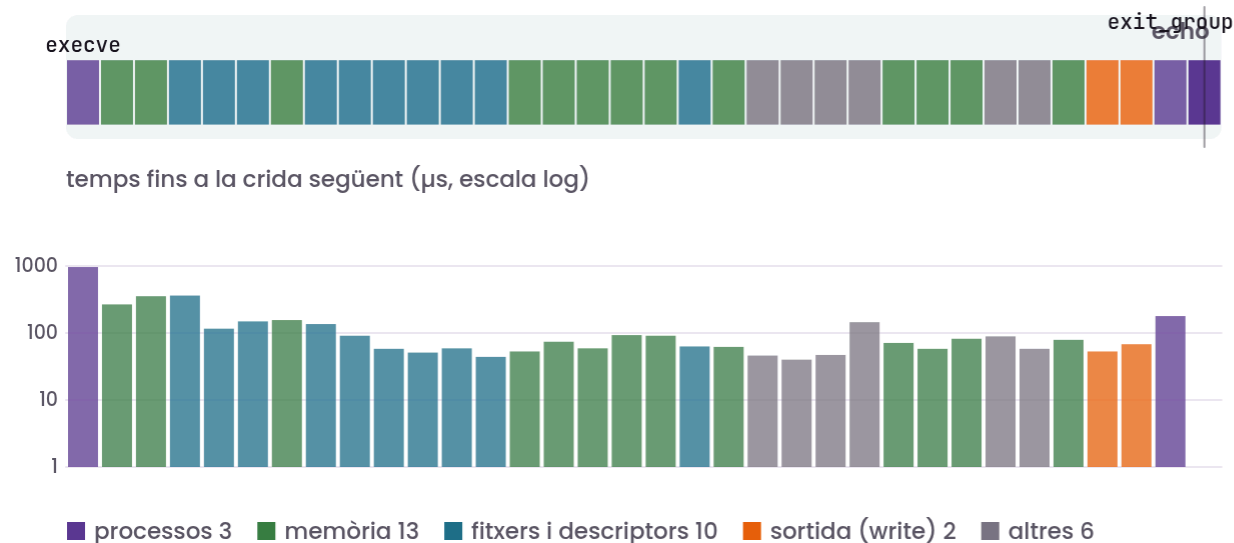
Sortida real

```
$ strace -r ./echo hola > /dev/null      # the whole life of the process
0.000000 execve("./echo", [".echo", "hola"], 0x7ffff5a76120 /* 129 vars */) = 0
0.000965 brk(NULL)                      = 0x615cdaf80000
0.000267 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x747beb566000
0.000354 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
0.000362 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
0.000116 fstat(3, {st_mode=S_IFREG|0644, st_size=108847, ...}) = 0
0.000149 mmap(NULL, 108847, PROT_READ, MAP_PRIVATE, 3, 0) = 0x747beb54b000
0.000156 close(3)                       = 0
...
```

Sortida real

```
0.000089 getrandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_NONBLOCK) = 8
0.000058 munmap(0x747beb54b000, 108847) = 0
0.000079 write(1, "hola", 4)              = 4
0.000053 write(1, "\n", 1)                = 1
0.000068 exit_group(0)                  = ?
0.000179 +++ exited with 0 +++
```

Tota la traça, crida a crida



```
set_tid_address(0x747beb548d68) = 809641
set_robust_list(0x747beb548a20, 24) = 0
rseq(0x747beb548680, 0x21, 0, 0x53053053) = 0
mprotect(0x747beb40e000, 16384, PROT_READ) = 0
mprotect(0x615caa96a000, 4096, PROT_READ) = 0
mprotect(0x747beb5ab000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, ...
getrandom("\x70\x5a\xe6\xac\xf7\xee\x72\x1b", 8, GRND_...
munmap(0x747beb54b000, 108847) = 0
write(1, "hola", 4)      = 4
write(1, "\n", 1)       = 1
exit_group(0)           = ?
+++ exited with 0 +++
```

Crides al sistema i funcions de biblioteca

Crides al sistema (man 2)

Inodes: `stat`, `unlink`, `chmod`, `link` ...

Descriptors: `open`, `read`, `write`, `lseek`,
`close`, `fcntl` ...

Biblioteca de C (man 3)

`fopen`, `fread`, `fprintf`, `fclose` ... **no són crides al sistema**: fan servir `FILE *` (amb memòria intermèdia) en lloc d'un descriptor. `fileno()` converteix l'un en l'altre.

Llegir dades: `read`

C

```
ssize_t read(int fd, void *buf, size_t count);
```

- Llegeix **fins a** `count` bytes del fitxer de `fd` a `buf`, que ja ha d'estar assignat (`count` $\leq$ la seva mida)
- Retorna els bytes llegits, **0 al final del fitxer** o -1 en cas d'error
- **Pot retornar menys** bytes dels demanats: canonada, terminal, xarxa o un senyal (`EINTR` si no ha llegit res)

Escriure dades: `write`

C

```
ssize_t write(int fd, const void *buffer, size_t count);
```

- Escriu els primers `count` bytes de `buffer` al fitxer de `fd`
- Retorna `-1` en cas d'error o els bytes escrits

Correcció a l'original: «si escriu menys de `count`, és un error» no sempre és cert. Amb canonades o sòcols plens, o un senyal a mig camí, l'escriptura pot ser **parcial i vàlida**: repeteix amb la resta (`write_all`).

On és l'error?

C

```
/* slide 51 of the original */
while ((byte = read(fd, buf, length))  $\neq$  0) {
    if (byte == -1) {
        perror("read() failed"); exit(1);
    }
    length = length - byte;
    buf = buf + length;      /* BUG */
}
```

C

```
/* fixed: advance by what was READ */
while (length > 0) {
    byte = read(fd, buf, length);
    if (byte == -1 && errno == EINTR)
        continue;
    exit_if(byte == -1, "read");
    if (byte == 0)           /* end of file */
        break;
    length -= byte;
    buf += byte;
}
```

El mateix bucle amb un fitxer i amb una canonada

Original (diapositiva 51)

```
while ((byte = read(fd, buf, length)) != 0) {
    if (byte == -1) {
        perror("read() failed"); exit(1);
    }
    length = length - byte;
    buf = buf + length;
}
```

buf = 43 length = 0

\$ strace -e trace=read ./prog (pipe)

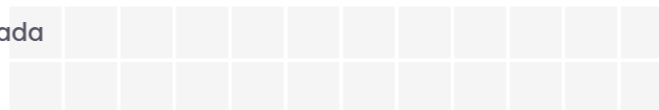
read(3, buf+36, 16) = 9

read(3, buf+43, 7) = 7

read(3, buf+43, 0) = 0

canonada: l'escriptor fa 4 write() de 4 + 4 + 9 + 7 bytes

canonada



memòria del procés: char buf[24] i el que ve després

+0 r e a d

+12 () m

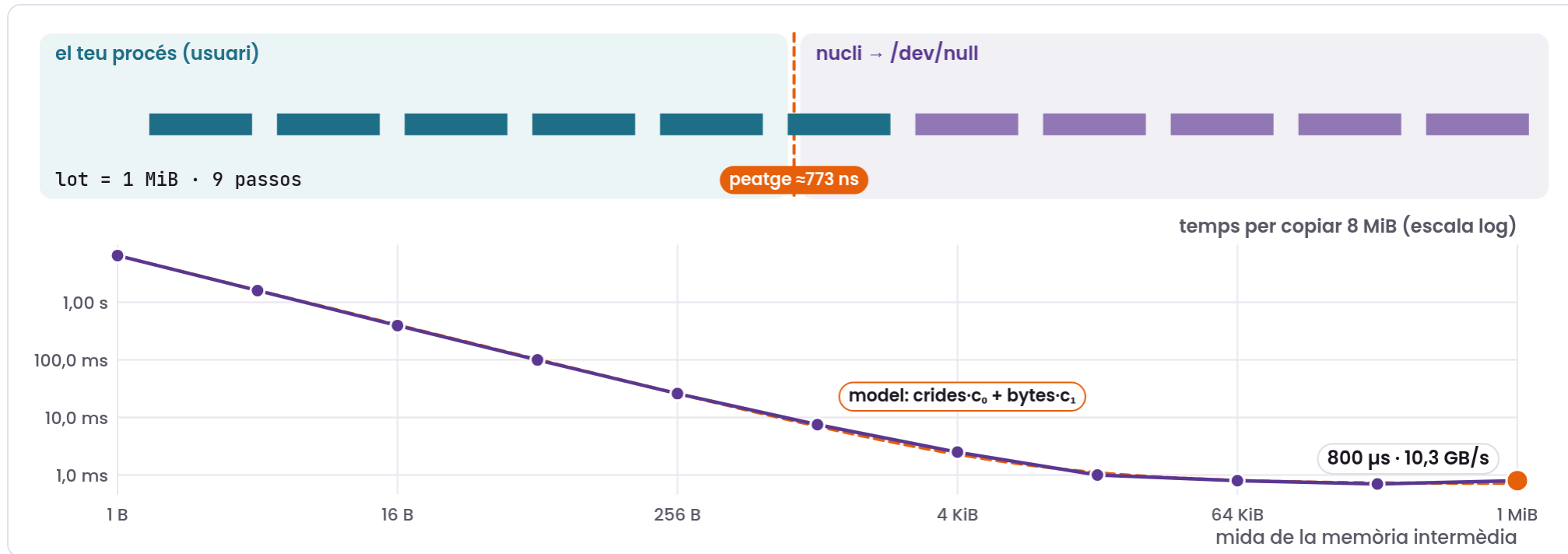
final de buf[24] · memòria veïna: altres variables, adreça de retorn...

+24 ? ? ? ? ? ? ? ? ? ? ? ?

+36 a y r e t u l e s s

buf = 43

Quant costa cada pas de frontera? Mesurat



Menys passos de frontera, la mateixa feina

vDSO

`clock_gettime` sense entrar al nucli.

io_uring

Cues compartides:
centenars d'operacions
per crida.

eBPF

Observar les crides a tot
el sistema sense aturar
processos.

TD1 resol't + repte IA

- Compila i executa els 7 exercicis; trenca'n un a propòsit
- Explica cada línia de `strace ./echo`
- Repte IA: mesura el cost de les crides a la teva màquina

Bash

```
make -C tds && tds/tests/run.sh td1
```



Universidad
Indoamérica

PER EMPORTAR-TE

Tres preguntes

1. Per què el nucli no és un procés, i per què no és root?
2. Què retorna `read` al final del fitxer i què fa glibc amb -9?
3. Què fallava a l'exemple de la diapositiva 51?